Fractionally Differentiated Features

Adding a Notion of Memory to a Dataset

Last Time

- Our main objective was to remove both the noise and notion of *time* from our dataset.
  - This removed values that did not really tell us anything(noise)
  - This removed a false structure from our dataset, so our model doesn't accidently believe time is a direct indicator of future performance.
    - It's like a child, we don't want to teach it the wrong lessons
- The dollar bars properties are invariant by the change of the origin time
  - They sample the market at a constant rate of informational arrival(volume/dollars) rather than clock time

Let’s Refocus Our Goal

- We want:
  - Agentic decision making
  - The ability to act based on the sentiment of the market
  - Money
    - (profitable trading)
- Everything we have done so far is building the *foundations* to make that happen
  - Dollar bars were the first step in this path
- To understand our next step, we need to understand how we go from *dataset* to *decision*
  - This happens through a process called supervised learning

Supervised Machine Learning

- Supervised learning is a *category* of machine learning that uses labeled datasets to train algorithms to predict outcomes and recognize patterns.
  - We provide labeled training sets and hope to *learn the relationships between the input and outputs*
    - Inputs: **features**
    - Corrected outputs: **labels**
- Algorithms analyze a large dataset of these training pairs to infer what a desired output value would be when asked to make a prediction on new data

The Engine Under the Hood

- Pattern Recognition:
  - Supervised ML is essentially a high-powered version of *connect the dots*
    - It looks at our **features(**dollar bars, RSI, volatility, market cap, whatever we choose to label as a *feature*) and tries to find a mathematical function that maps them to a **label(**buy,sell,hold)
- Stages
  - The Training phase
    - The model looks at thousands of post examples
      - Every time, it asks: if the price dropped by 2% and volume spiked, what happened next?
  - The Inference phase
    - When a model sees the same pattern in live data, it applies the rule it learned from the past to predict the future
- In order for this to work, the “rules” of the market must stay relatively consistent
  - If the environment changes too much, the patterns the model learns from becomes useless
    - ….but wait, isn’t the market all about rapid environment changes?

Memory is Predictive Power

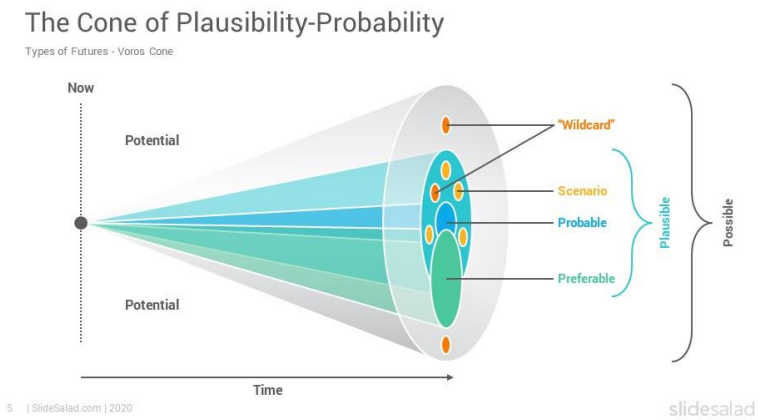
- Models need some notion of memory to draw signals of performance from
  - “Trend is your friend”
- This raises a concern with our raw dollar bars
  - We have a raw series of prices ($150, $153, etc)
  - Day X in the dollar bar data has **zero** impact on Day X + 1
    - If we train a model on prices from 2020 and test in on 2024, the model will fail because the statistical properties(range of values) is completely different
    - Ultimately the statistical properties (mean, variance) do not change over time. If you train a model on past data, you assume the future looks statistically similar.

Dollar Bars Examined

- We **lobotomized** our data. Dollar bars have **zero** notion of history
  - For each bar, the current price P_t is the sum of **all** past changes
    - Consider:
      - imagine a "Drunk Walker" leaving a bar (the starting price, P_0)
      - Every minute he takes one step
      - He flips a coin to decide: heads -> north, tails-> south
        - These are our **returns** or changes
    - A price is a “cumulative sum”
      - If we want to know where the walker is 100 minutes after he leaves the bar, you have to add up every single step he took since leaving the bar
      - Position at Minute 1 = Start + Step 1
      - Position at Minute 2 = Start + Step 1 + Step 2
      - Position at Minute 100 = Start + Sum of all 100 Steps
    - The above logic is how *graphs* are charted

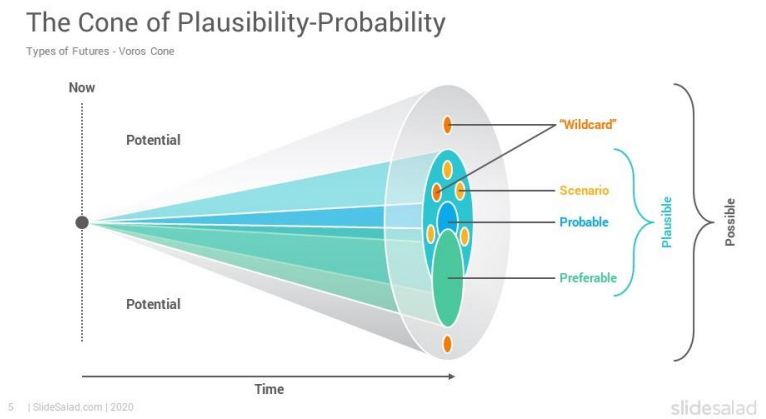
Visualizing the "Spread" of Uncertainty

- When the walker takes a single step
  - The “cloud” of where he might be is tiny
    - There are only two possible spots(north or south)
- Let’s consider
  - What is the probability of a coin flip?
  - For our walker, what does that mean?
  - (mean)If we flip a coin 100 times, 1000 times, 10,000 times
    - Where do we estimate the most likely place for him to be is?
  - (tails) What happens to the extremes in this case?

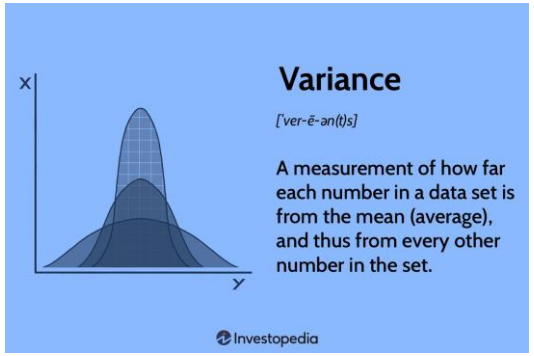


Probability Visualized

- As the drunk takes more steps, the number of places he *could* be expands.
  - While he is still *statistically likely* to be at the center(mean), the walls of where he might be spread out as a factor of time ($\sqrt{t}$)
- The center line represents the “expected value”
- The cone is our *variance*
  - Notice, it starts tight and expands as *t grows*



Understanding Variance



- Variance measures **how far a set of numbers are spread out from their average value(mean)**
- Each new step is random, it adds a new layer of uncertainty
  - We add the uncertainty of step 100 on top of the uncertainty of step 99, on top of 98, etc.
- Mathematically, if the variance of one step is σ^2, the variance after t steps is $t \times \sigma^2$
  - Because the variance equals $t \times \sigma^2$, as time (t) goes to infinity, the variance goes to infinity. This presents the **wandering problem**(infinite variance).

What do the “Math Police” Want?

- For Machine learning to work, our model needs to **trust** our data
  - We can’t allow this idea of *wandering or infinite variance*
- Does anyone remember what RSI tells us?
  - Imagine we train an AI to “buy” whenever the RSI is low and the price is $100
    - In 2004, $100 might be an all-time high (A "Sell" signal)
    - In 2024, $100 might be a massive crash bottom (A "Buy" signal)
  - Because the price (the walker) has "wandered" so far from its 2004 starting point, the **number 100** no longer means the same thing to the model.
- Our goal is to create **stationary** data, let’s formalize the rules

What makes something Stationary

- For a dataset to be considered stationary, we have three rules
  - 1. The average value shouldn't drift over time
    - **Price Fails:** If you look at NVDA in 2015, the average price was maybe $5. In 2024, the average price was $900. The mean has drifted massively
      - **AI get's completely lost trying to compare the data**
  - 2. Constant Variance: The range of the dataset should not expand or contract wildly
    - **Price Fails:** When a stock trades at $100, a "big day" might be a $2 move. When a stock trades at $1000, a "big day" is a $20 move.
      - As price grows, the variance(volatility in dollar terms) grows with it. The technical term for this is *heteroscedasticity*
  - 3. No Memory (Independence): Knowing yesterdays value shouldn't automatically tell you todays value
    - **Price Fails:** Prices have a long memory. If the price closed at $500 yesterday, your best guess for today is $500. This is heavily dependent on the past.
    - Requires us to sum *every* step taken in history

Stationary is Stability

- **Mean Stability:** The "average" shouldn't drift. If your drunk walker is in a hallway, the mean is the center of the hall. It doesn't move.
- **Variance Stability:** The "width" of the possible moves stays the same. The walker doesn't get more erratic the longer he walks.
- **Covariance Stability:** The relationship between two points in time (e.g., today vs. 5 days ago) only depends on the *distance* between them, not the *actual date*.

Stationary vs Non-Stationary

- Our current reality is dollar bars that are **non-stationary**
  - They behave like a drunk walker. If he walks for a million years, he could end up on the other side of the galaxy
  - We never **reset** the uncertainty
    - A shock that happened in 2022 is still included in the price calculations for 2024
      - This accumulation of history causes the range of possible values to expand indefinitely as time passes
- Our goal is to create **Stationary Data**
  - We allow the data to move around, but the variance is limited to staying closer to the mean
    - It's like putting the drunk guy in a child leash as we lead him home.

Data Cleaning Continued

- Our goal is *once again* to transition this data into more useable information through a process known as *data cleaning*
  - Reminder
    - Data is raw, chaotic, messy
    - Information is refined signals that a model can use to make a decision
- We have gone through two layers of filtering so far
  - Ticks -> OHCVL -> Dollar bars
  - All assets on NASDAQ + NYSE -> top 1000
- We need *another* layer
  - We want to transform our data into a state that satisfies the “math police” without losing the **story** the market is telling us

Cleaning Data for a "Reasonable" Model

- Imagine you are training an AI to control a **Smart Thermostat**
  - The Raw Data (Non-Stationary):
    - The raw sensor reading is 72(F)
      - What is the context(the *external environment*)?
  - The cost of combating Drift
    - Imagine the outside temperature is 110.
      - If you only look at the room temperature, the data looks "fine." It looks stable.
      - In reality the AC might be ready to blow a fuse because it's under immense pressure
      - Our AC unit struggles because it is fighting an **unbounded process** using a **bounded tool**
    - This ability to combat drift is incredibly important
      - If a stock stays at $150 for two years, while the rest of the market crashes around it
        - This isn't *sideways data,* it's an incredible signal of stability and strength
- In non-stationary systems, the "absolute value" (Price or Temperature) hides the true story.
  - We clean the data so the model can see the **relative struggle**, not just the final number.

The Drunk Walker in a New Suit

- He looks better, he's moving more logically based, but he's still drunk
  - We have removed the clock time noise
    - We are now sampling based on **economic activity**
    - But…the result is still a series of absolute prices
- The lingering problem
  - Even if the walker only takes a step at a time, he’s still drunk!
- Our cleaned dollar bars tell the model: “the price is $150!”
  - The model asks: “is that expensive?”
- Reimagining:
  - We need to stop looking at the dollar bar as the **destination**(price)
  - We need to *start* looking at it as the **Signal**(change) that remembers it’s path

The Industry Standard

- Dollar Returns
  - This allows us to convert our absolute price into a series of percentage changes
  - This is the percentage change from day X to day X+1
    - Return=(Price Today−Price Yesterday) / Price Yesterday
  - In math terms:
    - Raw price is a cumulative sum; it carries the baggage of every trade ever made
    - By subtracting yesterday's price, we are **differentiating** (taking the difference)
    - We are deleting the cumulative history and leaving only the **innovation**
- It allows us to create a **delta** Δ that shows the change from day x to day x+1
  - +2%, -2%, +5%, -10%
- Let’s run this through our three rules to see how the outcome shifts

Stationary Check: Returns

- 1. The average value shouldn’t drift over time
    - **Returns passes:** If NVDA goes up 1% in 2015, and 1% in 2024, those numbers are mathematically identical (0.01). Whether the stock is trading at $5 or $900, the daily percent change usually hovers around an average of 0.0% to 0.05%. This results in a stable mean
- 2. Constant Variance: The range of the dataset should not expand or contract wildly
    - **Returns passes:** By converting to percentage terms, you normalize this.
        - A $2 move on $100 is 2%. A $20 move on $1000 is also 2%. The "scale" of the data stays consistent regardless of the price level
- 3. No Memory (Independence): Knowing yesterdays value shouldn’t automatically tell you todays value
    - **Returns passes:** This is where you "strip the memory." If the stock went up 1% yesterday, that tells you almost *nothing* about whether it will go up or down today.
        - Returns are largely independent noise. You have deleted the history of where the walker came from and are looking only at his last step
- Back to our dunk person example:
  - (dollar bars) you get a phone call from the walker and he says: “I am at mile marker 50”
    - Prediction: our best guess is his next step has him somewhere near mile marker 50
  - (returns) you get a phone call from the walker and he says: “I just took a step north”
    - Prediction: Does knowing he stepped North today tell you where he is on the map?

It is **never** that easy!

- Although stationary data is a necessary property for inferential purposes and machine learning, it is rarely the case in signal processing that we wish all memory to be erased
  - At the end of the day, memory is the basis for the model's predictive power
- As an example:
  - Equilibrium models need some memory to assess how far the price process has drifted away from the long term expected value to generate a forecast
- Our Delema
  - Returns are stationary, however memory-less, and prices have memory, however they are non-stationary

Issues to consider

- We know supervised learning algorithms typically require station features
  - They reason is that we need to map a previously unseen(unlabeled) observation to a collection of labeled examples, and infer from them the label of that new observation
  - If our features are not stationary, we cannot map the new observation to a large number of known examples
  - We know that stationary does not ensure predictive power
- Stationarity is a necessary, non-sufficient condition for the high performance of a ML algorithm

The fundamental trade off

- We are stuck at this trade-off between stationarity and memory
  - We can always make a series more stationary through some mathematical operations but it comes at the cost of erasing some memory
    - Defeats the purpose of the ML algorithm!
- This leads us to the fundamental question of
  - What is the **minimum** amount of differentiation that makes a price series stationary while preserving as much memory as possible?
  - How do we go about calculating this, how do we go about verifying this?

Why We Usually Fail

- Standard finance forces us to choose integers
  - The history has trained sheep to only think of two choices
    - At d=0, the weights applied to history are effectively just 1.0 on the current observation. We treat the price as a "cumulative sum" of every shock that has ever happened
    - At d=1, We calculate $Pt-Pt-1$. In terms of weights, we apply a weight of 1.0 to today and −1.0 to yesterday. The weight for the day before yesterday ($t-2$) is **0**. The weight for last year is **0**. All history beyond 1 lag is mathematically erased
- The Engineering question
  - Is there somehow a number that can exist between 0 and 1
  - Choosing d=1 is mathematically arbitrary. Why must we subtract *exactly* 100%
    - Why not 40% from yesterday, 10% from two days…so on

Fractional Differentiation

- Instead of setting the differentiation to either 0 or 1, we treat the differentiation order D as a **continuous variable**
  - Ex: what happens if we differentiate by d=0.4
- This process is called **Fractional Differentiation**
  - We use a binomial series expansion to distribute the subtraction over a long history
- The Result
  - d=0 (Free Walk): The drunk walker wanders infinitely away (Variance explodes).
  - d=1 (Reset): The walker is teleported back to the start every minute (Memory erased).
  - d=0.4 (Bungee Cord): The walker can move and establish a trend, but the weights (the tension) slowly pull the variance back towards the mean.

The Hidden Drift

- The Binomial Expansion technically requires an infinite history of prices to calculate the correct value for today
  - We don’t have that, at most we have our start time of ‘22 as an example, but don’t have any data before it
- So, what do we do?
- Most people solve this by using an **"Expanding Window”**
  - Using all available data from the start of the file up to today.
    - Time 100: You apply 100 weights to calculate the value.
    - Time 1,000: You apply 1,000 weights to calculate the value.
    - As time goes on, you are adding more and more negative weights to the calculation.
      - Remember that the weights (ω) are negative (e.g., -0.4, -0.12).
- This causes the resulting values to mechanically drift downward over time, purely because the window is getting longer, not because the price is changing.

Fixed-Width Window

- Our expanding window failed because the memory length kept changing(growing) causing the values to shift
- To fix this, we need to force memory to have a constant lifespan
  - We need some sort of window…that slides
    - A sliding window perhaps!
- The algorithm
  - We set a **tolerance (τ)**, typically 1e-5
  - We calculate the weights until they drop below the threshold
  - If weight #500 is smaller than τ, we stop there
- Result:
  - We have a stationary feature that preserves the trend without carrying the infinite baggage of the past

The Optimization Search

- Our goal
  - We want to find the smallest possible differential value that satisfies the math police(stationary)
    - We can automate this process using the augmented Dickey-Fuller test
- The **Augmented Dickey-Fuller Test:**
  - This is the standard statistical test used to determine if a time series is stationary or if it behaves like a random walk
- The Algorithm
  - Transform the dollar bars using the current D value
  - Calculate the p-value(result of the above test)
    - Check if the result is less than 0.05
      - NO: the series is still non-stationary, increase D
      - Yes: Stop, you found the sweet spot!
- At this specific point, we satisfy the "Math Police" (Stationarity) while keeping ~95% correlation with the original price history (Maximum Memory)
  - Emphasize that we could just use d=1 (standard returns) and pass the test easily. The entire point of this loop is to do the minimum amount of damage to the data necessary to pass the test

The Implementation Task